C Creativo

Penjelasan Materi
Week 1–6

Pemrograman Mobile: materi tiap pertemuan, penerapannya di aplikasi Creativo, dan hubungan syntax dari slide ke kode yang sebenarnya.

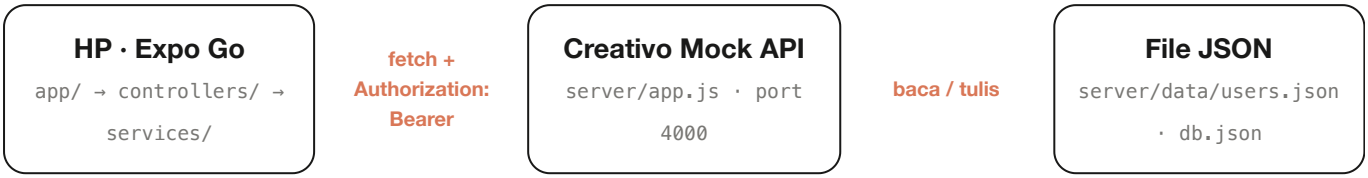
Gambaran besar	Arsitektur, cara menjalankan
Week 1	Course Orientation & Project Kickoff
Week 2	From Idea to Mobile Interface
Week 3	From Interface to Secure Mobile App
Week 4	From Interactive to Data-Driven
Week 5	From API to Secure API
Week 6	From Successful API to Reliable API
Demo	Persiapan, urutan presentasi, skrip, rencana cadangan
Lampiran	Akun dummy, penyimpanan data, hasil tes

Mata kuliah Pemrograman Mobile · Semester 5

21 Oktober 2026

Gambaran besar

Creativo adalah aplikasi Expo / React Native tempat profesional memamerkan karya, dan klien atau recruiter mencari kandidat lewat karya nyata. Mulai Week 6, Supabase ditunda: semua data lewat **Creativo Mock API** di folder `server/`, sehingga alur register, login, token, dan protected API sama persis dengan modul.



Week	Materi	Di Creativo	Tes
1	Kickoff, product canvas, GitHub	Product canvas, repo, template PR/issue, CI	CI
2	UI, responsif, aksesibilitas	Beranda, tab, landscape, ukuran teks, label/role/hint	—
3	Auth, secure storage, security test	Daftar → Masuk → Beranda, biometrik, SecureStore vs AsyncStorage	security.test.ts
4	Fetch API, JSON, loading, error	Lowongan dari Himalayas API	api.test.ts
5	Token, JWT, Bearer, protected API	Mock API, interceptor `api()`, logout mencabut token	token.test.ts
6	Status code, timeout, offline, retry	ErrorState per jenis error, useRequest, OfflineBanner, Uji Kondisi API	reliable.test.ts

Menjalankan

```
npm install
npm run server      # terminal 1: API di port 4000
npx expo start      # terminal 2: scan QR dengan Expo Go (HP satu Wi-Fi dengan laptop)
npm test            # 68 tes Week 3–6 (tanpa server, tanpa internet)
```

> Face ID tidak bisa dipakai di Expo Go untuk iOS (batasan Expo Go); gunakan development build. Sidik jari Android bisa langsung di Expo Go.

WEEK 1

PERTEMUAN 1 · COURSE ORIENTATION & PROJECT KICKOFF

Dari masalah menjadi aplikasi

Belum coding: menentukan pengguna, masalah, solusi, dan fitur utama, lalu menyiapkan GitHub sebagai tempat kerja tim.

Inti materi

Pemrograman mobile tidak dimulai dari fitur, tetapi dari **masalah pengguna**. Alurnya **Ide → Design → Build → Mobile App**, dikerjakan sebagai tim developer secara berulang (**Backlog → Sprint → Build → Test → Demo → Improve**). Selama semester, aplikasi tumbuh lewat empat bidang: **UI/UX**, **Security**, **API & Network**, dan **Device Features**. GitHub menjadi tempat kerja bersama dengan empat konsep dasar: **repository**, **commit**, **branch**, dan **pull request**. AI boleh dipakai dengan tiga checkpoint: **Understand** (paham kodenya), **Verify** (benar dan aman), **Modify** (bisa menyesuaikan).

Product Canvas Creativo

Pertanyaan	Jawaban
1 · Siapa penggunanya?	Profesional kreatif dan teknis (desainer, developer, fotografer, arsitek, kreator video, data scientist, marketer), serta klien dan recruiter yang mencari mereka.
2 · Masalahnya?	Karya tersebar di banyak platform dan CV tidak menunjukkan karya nyata. Klien sulit menilai kemampuan dan menghubungi kandidat.
3 · Solusinya?	Satu aplikasi untuk memamerkan karya dan membangun koneksi profesional. Klien mencari berdasarkan bidang dan status **Siap direkrut**, lalu langsung mengirim pesan.
4 · Aplikasinya?	**Creativo** — “Tunjukkan karyamu. Perluas relasimu.” Aplikasi Expo / React Native.
5 · Fitur utama?	Daftar & masuk yang aman · profil profesional · unggah karya · feed, suka, komentar · cari profesional · koneksi & pesan · lowongan kerja.

Hubungan materi dengan proyek

Konsep di slide	Di proyek Creativo
Repository	Repo `CREATIVO`: kode, dokumentasi (`README.md` , `Weekly Task/`), data seed server
Commit	Riwayat per minggu: `week 4` , `JWT login` , `jwt fix` , …
Branch & Pull Request	Template PR `.github/pull_request_template.md` (PR #1 “setup alur kerja tim”)
Backlog	Template issue `user-story.md` dan `bug.md`
Test	CI `.github/workflows/ci.yml`: lint, typecheck, dan `npm test` di setiap push
AI: Understand · Verify · Modify	Setiap perubahan dicek dengan `npx tsc --noEmit` , `npx expo lint` , `npm test` , lalu dicoba di Expo Go

WEEK 2

PERTEMUAN 2 · SPRINT 01

From Idea to Mobile Interface

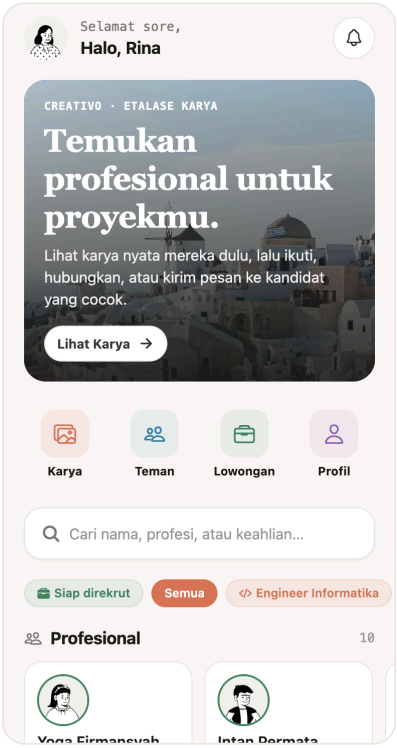
Membangun UI dengan komponen React Native, membuatnya responsif, dan inklusif untuk lebih banyak pengguna.

Inti materi

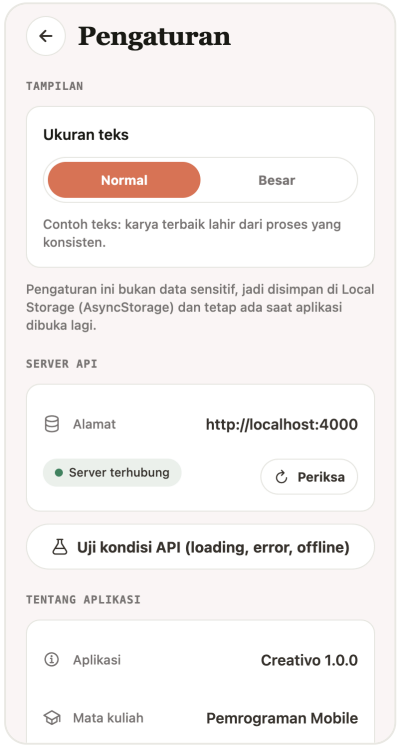
React Native membangun UI dari **component**: `View` (wadah), `Text`, `Image`, `Pressable` (bisa ditekan), diberi gaya dengan `StyleSheet` dan diatur dengan **Flexbox**. **Expo Go** menjalankan aplikasi langsung di HP. Tiga tugas sprint:

- **Task 01 · Build the Mobile UI**: halaman utama minimal punya header, judul, informasi utama, gambar, tombol aksi, dan bagian konten.
- **Task 02 · Responsive**: tetap rapi di layar kecil, normal, besar, portrait, dan landscape. Tidak ada yang keluar layar atau terpotong.
- **Task 03 · Inclusive & Accessible**: teks mudah dibaca, kontras cukup, target sentuh nyaman, dan dikenali screen reader lewat `accessibilityLabel`, `accessibilityRole`, `accessibilityHint`.

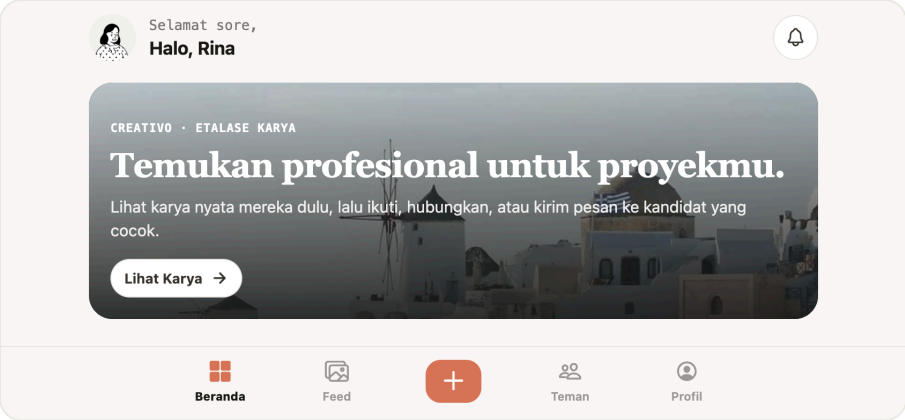
Di Creativo: Beranda



Beranda: header, judul, gambar, tombol, menu, konten



Pengaturan: ukuran teks Normal / Besar



Landscape: tata letak menyesuaikan lebar layar

Elemen minimal	Di Beranda (src/app/(tabs)/home.tsx)
Header	Avatar, “Selamat sore, **Halo, Rina**”, tombol notifikasi (baris 110–127)
Judul + informasi utama	“Temukan profesional untuk proyekmu.” dan penjelasannya (142–146)
Image / visual	Foto karya terbaru sebagai latar banner + gradasi agar teks putih terbaca (133–136)
Button / action	**Lihat Karya** → dan menu cepat Karya · Teman · Lowongan · Profil (147–176)
Content section	Cari, filter bidang, Profesional, Karya terbaru (178–248)
Navigation	Tab bawah Beranda · Feed · + · Teman · Profil (`views/navigation/TabBar.tsx`)

Hubungan syntax

Component React Native

Di slide (PresidentKu)

```
import { View, Text, Image, Pressable } from 'react-native';
```

Di Creativo `src/app/(tabs)/home.tsx:14`

```
import { Pressable, RefreshControl, ScrollView, StyleSheet, useWindowDimensions, View } from 'react-native';
```

Tombol aksi yang aksesibel

Di slide (PresidentKu)

```
accessible={true}
accessibilityLabel="Lihat Program"
accessibilityRole="button"
accessibilityHint="Membuka halaman program prioritas"
```

Di Creativo src/app/(tabs)/home.tsx:147–155 · role "button" di src/views/ui/Button.tsx:63

```
<Button
  label="Lihat Karya"
  iconRight="arrow-forward"
  size="sm"
  variant="secondary"
  accessibilityHint="Membuka feed karya"
  onPress={() => router.navigate('/feed')}
  style={styles.bannerButton}
/>
```

Responsive: jumlah kolom mengikuti lebar layar

Di slide (PresidentKu)

```
// Layout yang responsif: menyesuaikan ukuran layar
// dan orientasi (portrait / landscape)
```

Di Creativo src/app/(tabs)/home.tsx:56–64, 238 · app.json: "orientation": "default"

```
/** Karya per row: 2 on a phone, more on a tablet or in landscape */
function gridColumns(width: number) {
  const content = Math.min(width, CONTENT_MAX_WIDTH) - SCREEN_PADDING * 2;
  return content >= 600 ? 4 : content >= 440 ? 3 : 2;
}


const { width } = useWindowDimensions();
// …
<PostGrid posts={works} columns={gridColumns(width)} authors={profiles} />
```

StyleSheet + Flexbox

Di slide (PresidentKu)

```
const styles = StyleSheet.create({
  container: { flexDirection: 'row' },
});
```

Di Creativo src/app/(tabs)/home.tsx:266–274

```
topBar: { flexDirection: 'row', alignItems: 'center', gap: spacing.sm },
menu: { flexDirection: 'row', gap: spacing.xs },
menuItem: { flex: 1, alignItems: 'center', gap: 6, paddingVertical: spacing.xs, borderRadius:
radius.md, minHeight: 72 },
menuIcon: { width: 48, height: 48, borderRadius: radius.md, alignItems: 'center', justifyContent:
'center' },
menuLabel: { fontWeight: '700' },
```

Checklist Sprint 01

- ✓ React Native project berjalan
- ✓ Component digunakan dengan tepat
- ✓ Readable text (ukuran teks)
- ✓ Accessibility label
- ✓ Test pada perangkat
- ✓ Expo Go menjalankan aplikasi
- ✓ Responsive layout
- ✓ Contrast diperhatikan
- ✓ Accessibility role
- □ Commit
- ✓ Mobile UI dibuat
- ✓ Adaptive layout (portrait & landscape)
- ✓ Minimum touch target (≥ 44 pt)
- ✓ Accessibility hint
- □ Push

Pertanyaan demo

Apa yang dibuat?

> Beranda Creativo: etalase karya dan profesional dari sudut pandang klien.

Component apa yang digunakan?

> View, Text (lewat AppText), Image (expo-image), Pressable, ScrollView, FlatList, TextInput, StyleSheet.

Bagaimana layout menyesuaikan layar?

> Flexbox, `useWindowDimensions` untuk jumlah kolom, `maxWidth` 720 agar rapi di tablet, dan landscape diizinkan.

Apa yang membuat interface inklusif?

> Teks bisa diperbesar, kontras tinggi, target sentuh besar, dan pesan tidak hanya lewat warna (selalu ada teks dan ikon).

Bagaimana accessibility diterapkan?

> Role dan label di setiap tombol, label yang terlihat di setiap input, dan pesan error diumumkan ke screen reader.

WEEK 3

PERTEMUAN 3 · SPRINT 02

From Interface to Secure Mobile App

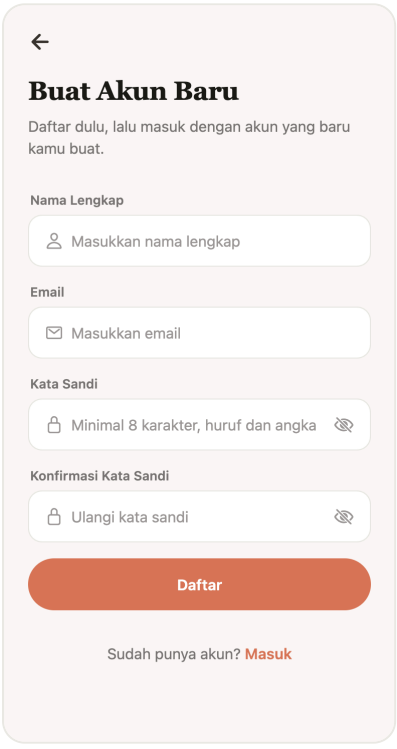
Autentikasi pengguna, perlindungan data lokal, dan pengujian keamanan.

Inti materi

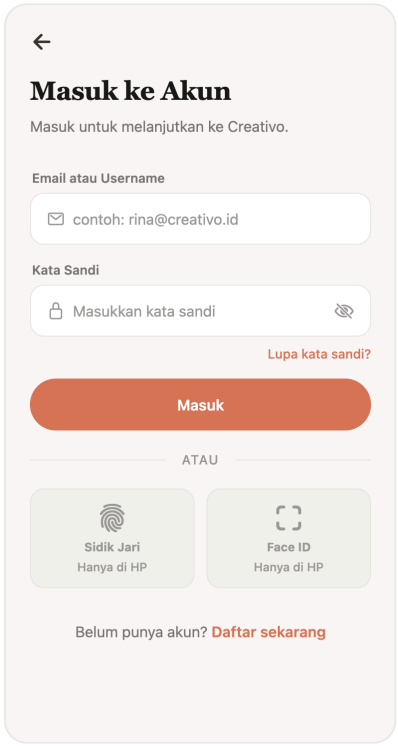
Authentication memastikan siapa penggunanya: **User → Credential → Validation → Authenticated → App**. Halamannya **Register** (buat akun), **Login** (masuk dengan akun yang terdaftar), dengan validasi **password/PIN** atau **biometrik** (sidik jari / Face ID). **Validasi input dilakukan sebelum autentikasi** agar tidak ada request kosong dan pesan error lebih jelas.

Local data security: data biasa (tema, bahasa, pengaturan UI) boleh di local storage, tetapi data sensitif (token, kredensial, sesi) harus di **secure storage** (Expo SecureStore, terenkripsi Keychain/Keystore). Password tidak boleh disimpan sebagai plaintext. Terakhir, keamanan **diuji, bukan diasumsikan**: credential benar, credential salah, input kosong, logout, data sensitif.

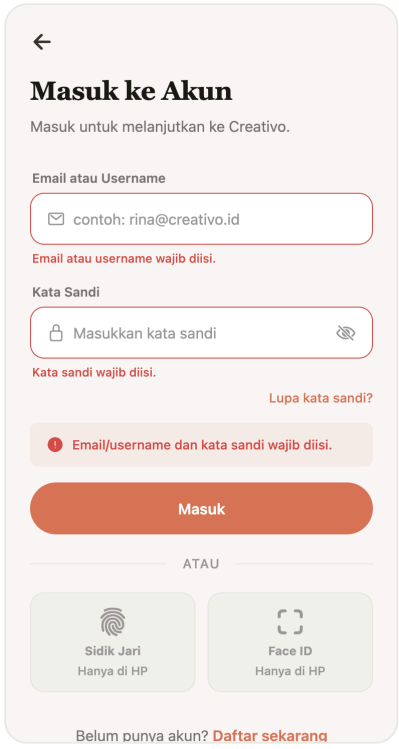
Di Creativo: Daftar → Masuk → Beranda



Daftar: nama, email, sandi, konfirmasi



Masuk: email/username, Sidik Jari, Face ID



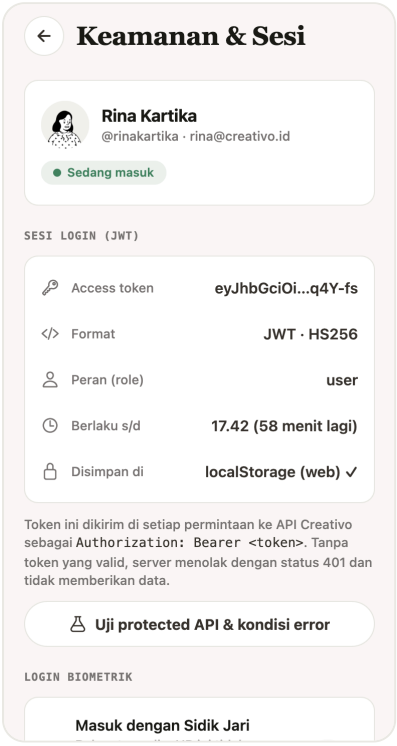
Input kosong → validation error

Daftar → `POST /auth/register` → **users.json (hash)** → "Akun berhasil dibuat" → **Masuk** → `POST /auth/login` → **token → SecureStore** → `isAuthenticated` → **Beranda**

- Akun hasil daftar disimpan di `server/data/users.json`. Kata sandi hanya disimpan sebagai **hash scrypt** (contoh: `scrypt$5b15…$4a1e…`).
- Login menerima **email atau username**. Kata sandi tersembunyi dengan tombol mata. 5 kali salah berturut-turut → dikunci 1 menit.

- **Login biometrik**: setelah masuk dengan kata sandi, aplikasi menawarkan “Masuk lebih cepat dengan Sidik Jari?”. Server memberi *token biometrik* yang disimpan di SecureStore. Saat menekan tile Sidik Jari/Face ID, HP memindai, lalu token itu ditukar dengan access token baru. Sidik jari atau wajah tidak pernah dikirim ke server.
- **Lupa kata sandi**: kode 6 angka (mode demo: tampil di terminal server) + kata sandi baru. Semua token lama ikut dicabut.

Data biasa vs data sensitif



Keamanan & Sesi (diambil dari versi web; di HP tertulis SecureStore)

Data	Jenis	Disimpan di
Access token (JWT)	Sensitif	SecureStore `access_token`
Token login biometrik	Sensitif	SecureStore `biometric_token`
Kata sandi	Sangat sensitif	Tidak disimpan di HP; di server hanya hash scrypt
Ukuran teks, tab feed terakhir, nama akun biometrik	Biasa	AsyncStorage (`creativo.textSize` , …)

Hubungan syntax

Login dengan validasi

Di slide (PresidentKu)

```
const handleLogin = () => {
  // 1. Validasi input terlebih dahulu
  if (!email || !password) {
    setError("Email dan password wajib diisi");
    return;
  }
  // 2. Cek credential (contoh sederhana)
  if (email === user.email && password === user.password) {
    setIsAuthenticated(true);
    setError("");
  } else {
    setError("Credential tidak valid");
  }
};
```

Di Creativo `src/app/(auth)/login.tsx:56–72`

```
async function handleLogin() {
  setError(null);
  // 1. Validation first: nothing is sent while the form is empty or malformed
  if (!validate()) {
    if (!identifier.trim() && !password) setError('Email/username dan kata sandi wajib diisi.');
    return;
  }
  // 2. Credential check by the server; on success the router leaves the auth screens by itself
  const result = await signIn(identifier.trim(), password);
  if (!result) return;
  if (result.error) {
    setError(result.error);
    setPassword('');
  } else if (result.profile) {
    offerAfterLogin(result.profile);
  }
}
```

Credential check: dilakukan server dengan hash, bukan teks biasa

Di slide (PresidentKu)

```
// Cek credential (contoh sederhana di lokal)
if (email === user.email && password === user.password)
```

Di Creativo `server/lib/password.js:20–26 · dipanggil di server/app.js:327`

```
function verifyPassword(password, stored) {
  const [algorithm, salt, hash] = String(stored ?? DECOY).split('$');
  if (algorithm !== 'scrypt' || !salt || !hash) return false;
  const expected = Buffer.from(hash, 'hex');
  const actual = crypto.scryptSync(String(password), salt, expected.length);
  return crypto.timingSafeEqual(actual, expected) && stored !== undefined && stored !== null;
}
```

Secure storage: simpan, baca, hapus

Di slide (PresidentKu)

```
import * as SecureStore from 'expo-secure-store';

await SecureStore.setItemAsync("auth_token", token);
const token = await SecureStore.getItemAsync("auth_token");
await SecureStore.deleteItemAsync("auth_token");
```

Di Creativo — `src/services/storage.ts:82–110 (kunci: 'access_token')`

```
export const secureStore = {
  /** The stored value, or null when there is none or it can't be read. */
  async get(key: string): Promise<string | null> {
    try {
      return isWeb ? (webStorage()?.getItem(key) ?? null) : await SecureStore.getItemAsync(key);
    } catch (err) {
      if (__DEV__) console.warn('[storage] secure read failed', err);
      return null;
    }
  },
  /** True once the value is saved. */
  async set(key: string, value: string): Promise<boolean> {
    try {
      if (isWeb) webStorage()?.setItem(key, value);
      else await SecureStore.setItemAsync(key, value);
      return true;
    } catch (err) {
      if (__DEV__) console.warn('[storage] secure save failed', err);
      return false;
    }
  },
  async remove(key: string): Promise<void> {
    try {
      if (isWeb) webStorage()?.removeItem(key);
      else await SecureStore.deleteItemAsync(key);
    } catch (err) {
      if (__DEV__) console.warn('[storage] secure delete failed', err);
    }
  },
```

Security test case → tes otomatis (npm test)

Di slide (PresidentKu)

```
const testCases = [
  { name: "Valid credential", expected: "LOGIN_SUCCESS" },
  { name: "Wrong credential", expected: "LOGIN_FAILED" },
  { name: "Empty input", expected: "VALIDATION_ERROR" },
  { name: "Logout", expected: "SESSION_CLEARED" },
  { name: "Sensitive data", expected: "PROTECTED" },
];
```

Di Creativo — `src/__tests__/security.test.ts`

```
describe('1. Credential benar → LOGIN_SUCCESS', …)   // baris 39
describe('2. Credential salah → LOGIN_FAILED', …)    // baris 66
describe('3. Input kosong → VALIDATION_ERROR', …)    // baris 90
describe('4. Logout → SESSION_CLEARED', …)           // baris 118
describe('5. Data sensitif → PROTECTED', …)          // baris 141
```

Checklist Sprint 02

- ✓ Login dibuat
- ✓ Register dibuat
- ✓ Credential validation
- ✓ Password/PIN validation
- ✓ Authentication flow
- ✓ Secure storage
- ✓ Sensitive data identified
- ✓ Sensitive data protected
- ✓ Logout
- ✓ Session cleared
- ✓ Security testing
- ✓ Accessibility tetap diterapkan
- ✓ Test pada perangkat
- □ Commit
- □ Push

Pertanyaan demo

Bagaimana proses login bekerja?

> Input divalidasi di aplikasi. Jika lengkap, dikirim ke `POST /auth/login`. Server mencocokkan hash kata sandi, lalu mengirim token. Token disimpan di SecureStore dan aplikasi masuk ke Beranda.

Bagaimana aplikasi mengetahui credential valid?

> Server menghitung hash scrypt dari kata sandi yang dikirim dan membandingkannya dengan hash di `users.json`.

Data apa yang dianggap sensitif?

> Access token, token biometrik, dan kata sandi.

Di mana data sensitif disimpan?

> Token di SecureStore (Keychain / Keystore). Kata sandi tidak disimpan di HP.

Apa yang terjadi saat logout?

> Konfirmasi muncul → token dicabut di server → dihapus dari SecureStore → kembali ke halaman Masuk.

Bagaimana menguji keamanan aplikasi?

> Lima skenario matriks dijalankan otomatis dengan `npm test` dan dicoba manual di aplikasi.

WEEK 4

PERTEMUAN 4 · SPRINT 03

From Interactive to Data-Driven

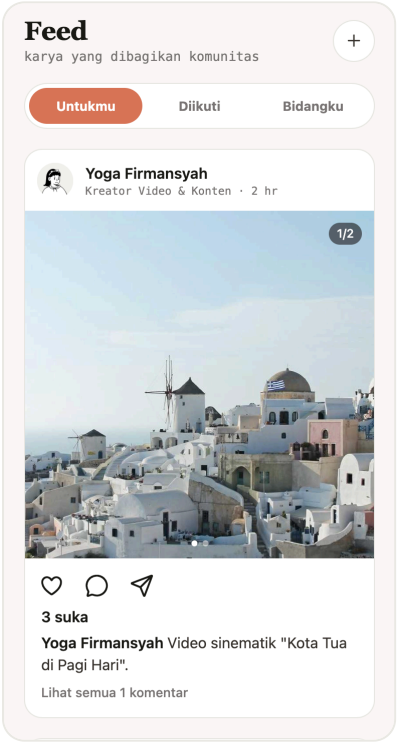
Mengambil data nyata dari REST API dengan fetch, mengolah JSON, dan menampilkannya.

Inti materi

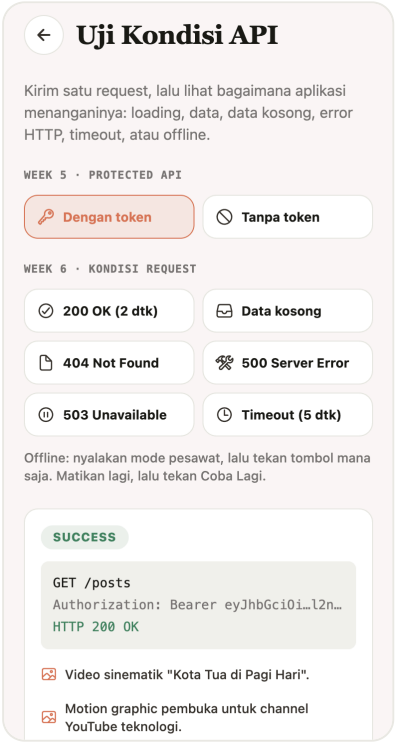
Aplikasi mengirim **request** ke **REST API**, server mengirim **response** berisi **JSON**, lalu data ditampilkan secara dinamis. Karena request butuh waktu, dipakai **asynchronous JavaScript**: **Promise** (`fetch(url).then(...)`) atau **async/await**. Data biasanya berupa **array of objects** yang dirender dengan perulangan. Saat menunggu tampil **loading state**; saat gagal, **error handling** dengan `try/catch` dan pengecekan `response.ok`.

Di Creativo

- **Lowongan**: lowongan kerja remote dari **Himalayas Jobs API** (`https://himalayas.app/jobs/api/search`, publik, HTTPS), sesuai profesi pengguna, dengan filter “Bisa dari Indonesia”. Status “Mengambil data… / Data berhasil dimuat / Gagal memuat data” dan tombol Coba Lagi.
- Sejak Week 6, **semua data aplikasi** (feed, profil, komentar, pesan) juga diambil dari REST API dengan pola yang sama.



Feed: array of objects dari API, dirender jadi kartu



Request berhasil: data tampil

Hubungan syntax

fetch + async/await + try/catch

Di slide (PresidentKu)

```
async function loadPrograms() {
  const url = "https://api.example.com/program";
  try {
    const response = await fetch(url);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Di Creativo — src/services/http.ts:143–167

```
let response: Response;
try {
  response = await fetch(url, init);
} catch (err) {
  // Network error: was it the phone's connection, or the server?
  return failure(apiError(abortKind() ?? ((await hasInternet()) ? 'network' : 'offline')), err);
}
    // …
if (response.status === 204) return { data: null as T, error: null };
try {
  return { data: (await response.json()) as T, error: null };
} catch (err) {
  return failure(apiError(abortKind() ?? 'invalid_json', response.status), err);
}
```

Error handling: response.ok

Di slide (PresidentKu)

```
if (!response.ok) {
  throw new Error("Gagal mengambil data");
}
```

Di Creativo — src/services/http.ts:152–160

```
if (!response.ok) {
  const kind = kindForStatus(response.status, detailedErrors);
  if (!detailedErrors) return failure(apiError(kind, response.status), `${response.status} ${url}`);
  // Our API explains what went wrong in the body: { message, code }
  const info = (await response.json().catch(() => null)) as { message?: unknown; code?: unknown } |
null;
  const message = typeof info?.message === 'string' ? info.message : undefined;
  const code = typeof info?.code === 'string' ? info.code : undefined;
  return failure(apiError(kind, response.status, message, code), `${response.status} ${method}
${url}`);
}
```

Render banyak data (array of objects)

Di slide (PresidentKu)

```
programs.forEach(program => {
  const card = `<article><h3>${program.nama}</h3></article>`;
  container.innerHTML += card;
});
```

Di Creativo `src/app/jobs.tsx:126–129`

```
<FlatList
  data={jobs}
  keyExtractor={(job) => job.id}
  renderItem={({ item }) => <JobCard job={item} onPress={openJob} />}
```

Loading state dan error state

Di slide (PresidentKu)

```
container.textContent = "Mengambil data...";
// …
container.textContent = "Gagal mengambil data";
```

Di Creativo `src/app/jobs.tsx:93–97`

```
const empty =
  status === 'loading' ? (
    <LoadingState />
  ) : status === 'error' ? (
    <ErrorState kind={error?.kind} message={error?.message} onRetry={reload} />
```

Checklist Sprint 03

- ✓ API request
- ✓ JSON
- ✓ JavaScript processing
- ✓ Dynamic data
- ✓ Loading
- ✓ Error handling
- ✓ Test (src/__tests__/api.test.ts)
- □ Commit
- □ Push

Pertanyaan demo

API apa yang digunakan?

> Himalayas Jobs API untuk lowongan; Creativo Mock API untuk data aplikasi.

Data apa yang diambil?

> Daftar lowongan remote: judul, perusahaan, gaji, lokasi, dan tautan lamaran.

Bagaimana data diproses?

> `fetch` → cek `response.ok` → `response.json()` → `parseJobPage()` membersihkan dan menerjemahkan data → state → `FlatList`.

Apa yang terjadi ketika request gagal?

> Tampil pesan error yang jelas dan tombol Coba Lagi. Jika data lama ada, data lama tetap tampil.

WEEK 5

SPRINT 04

From API to Secure API

Token-based authentication: login lewat API, JWT, Bearer token, protected endpoint, dan session.

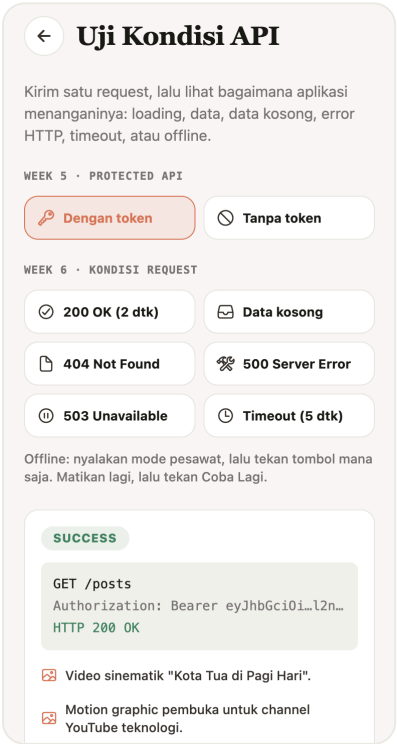
Inti materi

“API yang dapat diakses bukan berarti API yang boleh diakses siapa saja.” Setelah **login**, server mengirim **access token** berupa **JWT** (`header.payload.signature`). Aplikasi menyimpan token di **SecureStore**, lalu mengirimnya di setiap request ke **protected endpoint** lewat header `Authorization: Bearer <token>`, idealnya otomatis dengan **interceptor**. Server **memvalidasi** token (signature, masa berlaku, hak akses); tanpa token yang valid jawabannya **401**. **Logout** menghapus token dan mengembalikan aplikasi ke halaman login. Data harus dikirim lewat **HTTPS**.

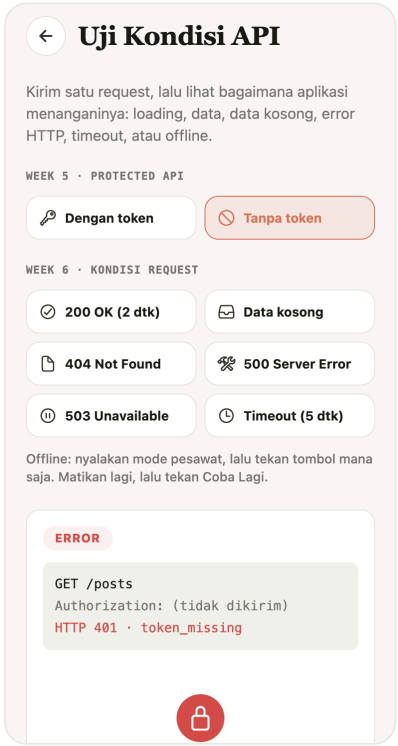
Login → POST /auth/login → **JWT** → setItemAsync → **SecureStore** → interceptor → **Bearer** → GET /posts → **200 / 401**

Di Creativo: Creativo Mock API

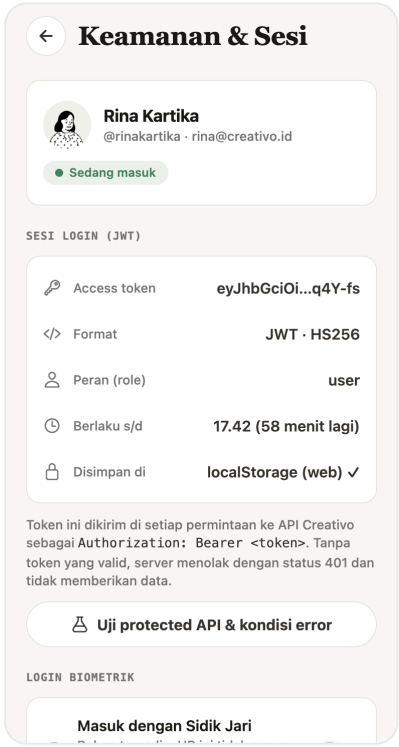
- `server/` : server Node.js tanpa dependensi (seperti `mock-server.js` di demo dosen). JWT HS256 dibuat dan diverifikasi sendiri.
- Semua data adalah **protected endpoint**. Token dipalsukan, kedaluwarsa (lebih dari 60 menit), atau sudah logout → 401.
- 401 di mana pun → token dihapus → kembali ke Masuk dengan pesan “Sesi kamu telah berakhir”.
- Logout mencabut token di server (id token disimpan di daftar `revoked_tokens`) lalu menghapusnya dari HP.



Dengan token → 200 OK



Tanpa token → 401 ditolak



JWT HS256, berlaku 60 menit, tersimpan ✓ (versi web)

Hubungan syntax

Login + simpan access token

Di slide (PresidentKu)

```
const response = await fetch("/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password }),
});
const data = await response.json();
await SecureStore.setItemAsync("access_token", data.access_token);
```

Di Creativo — src/services/auth.service.ts:38–69

```
export async function login(identifier: string, password: string): Promise<SessionResult> {
  logAuth('Login request → POST /auth/login');
  const result = await api<LoginResponse>('/auth/login', { method: 'POST', auth: false, body: {
identifier, password } });
  if (result.error) return { session: null, error: result.error.message, code: result.error.code };
  return startSession(result.data);
}

async function startSession({ access_token, user }: LoginResponse): Promise<SessionResult> {
  logAuth(`Response 200: access_token diterima → ${describeToken(access_token)}`);
  if (!(await secureStore.set(secureKeys.accessToken, access_token))) {
    return { session: null, error: 'Token login tidak dapat disimpan di perangkat ini.' };
  }
  logTokenSaved();
  return { session: { token: access_token, profile: user }, error: null };
}
```

JWT: header, payload, signature

Di slide (PresidentKu)

```
// JWT: header.payload.signature
{ "alg": "HS256", "typ": "JWT" }
{ "sub": "1", "name": "User", "role": "student", "exp": 1728144000 }
```

Di Creativo — server/lib/jwt.js:14–18

```
function signJwt(payload, secret) {
  const body = `${encode({ alg: 'HS256', typ: 'JWT' })}.${encode(payload)}`;
  return `${body}.${hmac(body, secret).toString('base64url')}`;
}


// server/app.js:149–152
const accessToken = signJwt(
  { sub: user.id, name: user.full_name, email: user.email, role: 'user', iat, exp, jti: uuid() },
  secret,
);
```

Interceptor: token otomatis di setiap request

Di slide (PresidentKu)

```
api.interceptors.request.use(async (config) => {
  const token = await getToken(); // dari secure storage
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Di Creativo — src/services/api.ts:69–75

```
export async function api<T>(path: string, { method = 'GET', body, auth = true, signal, timeoutMs,
poll = false }: ApiOptions = {}): Promise<ApiResult<T>> {
  const headers: Record<string, string> = { Accept: 'application/json' };

  // Interceptor: the token comes from SecureStore and rides along in the Authorization header
  const token = auth ? await secureStore.get(secureKeys.accessToken) : null;
  if (token) headers.Authorization = `Bearer ${token}`;
  if (poll) headers['X-Creativo-Poll'] = '1';
```

Logout: hapus token, akhiri session, ke halaman login

Di slide (PresidentKu)

```
await SecureStore.deleteItemAsync("access_token");
setIsAuthenticated(false);
router.replace("/login");
```

Di Creativo — src/services/auth.service.ts:77–89

```
export async function clearSession(): Promise<void> {
  await secureStore.remove(secureKeys.accessToken);
  logAuth('Token dihapus dari SecureStore → sesi berakhir');
}

/**
 * Logs out: the server is told first (it remembers the token as logged out, so a copy of it is
useless),
 * then the token is deleted from the device — also when the server can't be reached.
 */
export async function logout(): Promise<void> {
  logAuth('Logout request → POST /auth/logout');
  await api('/auth/logout', { method: 'POST' });
  await clearSession();

// useAuth.ts → setToken(null); index.tsx:20 →
if (!isAuthenticated) return <Redirect href={exitReason ? '/login' : '/welcome'} />;
```

Contoh log saat demo (terminal Metro)

```
[auth] Login request → POST /auth/login
[auth] Response 200: access_token diterima → Bearer eyJhbGciOi…q4Y-fs (JWT HS256 · berlaku 60 menit lagi)
[auth] Token disimpan ke SecureStore (terenkripsi) dengan kunci "access_token"
[api] GET /posts → 200 OK (6 ms) · Authorization: Bearer eyJhbGciOi…q4Y-fs
[auth] Logout request → POST /auth/logout
[auth] Token dihapus dari SecureStore → sesi berakhir
[auth] Authentication state → isAuthenticated: false, kembali ke halaman login
```

Checklist Sprint 04

- ✓ Token-based authentication
- ✓ Bearer token
- ✓ HTTPS (Himalayas; server lokal HTTP untuk development)
- ✓ Token validation
- ✓ Test authenticated request
- □ Commit
- ✓ Login melalui API
- ✓ Authorization header
- ✓ Secure token storage
- ✓ Logout
- ✓ Test unauthenticated request
- □ Push
- ✓ Access token
- ✓ JWT concept
- ✓ Protected endpoint
- ✓ Session cleared
- ✓ Test pada Expo Go

Pertanyaan demo

Dari mana access token diperoleh?

> Dari response `POST /auth/login` setelah server memverifikasi kredensial.

Di mana token disimpan?

> SecureStore dengan kunci `access_token` (lihat Keamanan & Sesi: “Disimpan di SecureStore ✓”).

Bagaimana token dikirim ke API?

> Fungsi `api()` membacanya dari SecureStore dan menambahkan `Authorization: Bearer <token>`.

Apa fungsi Bearer token?

> Bukti request berasal dari pengguna yang sudah login; siapa pun yang membawanya dianggap pemiliknya, maka harus aman dan lewat HTTPS.

Apa yang terjadi jika token tidak dikirim?

> Server menjawab 401 “Akses ditolak: token tidak ada” dan tidak memberikan data.

Apa yang terjadi ketika logout?

> Token dicabut di server dan dihapus dari HP; aplikasi kembali ke halaman Masuk; token lama tidak bisa dipakai lagi.

WEEK 6

SPRINT 05 · ERROR HANDLING & OFFLINE

From Successful API to Reliable API

Aplikasi tetap memberi feedback yang jelas saat request gagal: HTTP error, timeout, atau tanpa internet.

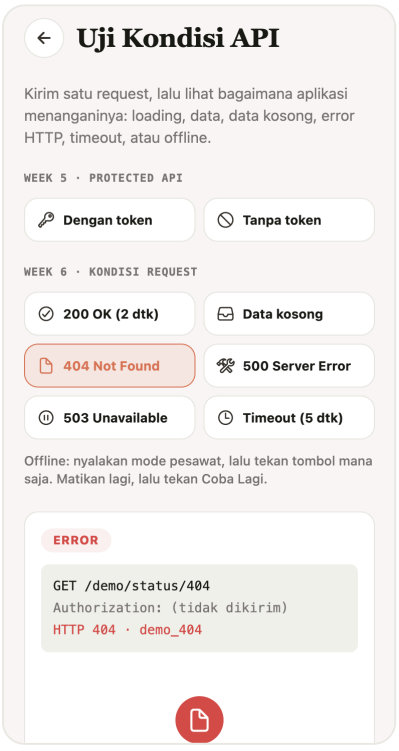
Inti materi

Tidak semua request berhasil, dan **tidak semua kegagalan berarti aplikasinya salah**. Empat kemungkinan hasil request:

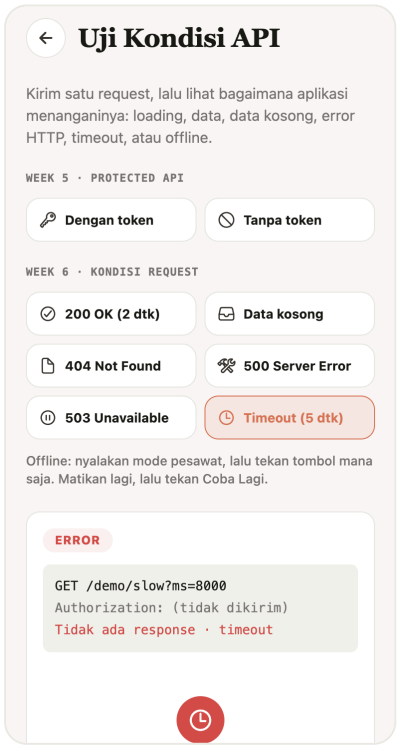
Hasil	Arti	Yang ditampilkan Creativo
Success (2xx)	Data diterima	Data, atau “Belum ada data” jika kosong
HTTP error (4xx / 5xx)	Server menjawab dengan error: 400 tidak valid, 401 belum login, 403 tidak punya izin, 404 tidak ditemukan, 500/503 server bermasalah	“Data tidak ditemukan”, “Terjadi kesalahan pada server”, … + Coba Lagi
Timeout	Server terlalu lama (batas 15 detik)	“Waktu habis” + Coba Lagi
No internet	HP tidak punya koneksi	Banner merah + “Tidak ada koneksi internet” + Coba Lagi

Setiap request punya **state UI**: **idle** → **loading** → **success / empty / error**. UI tidak boleh hanya punya kondisi “data ada”.

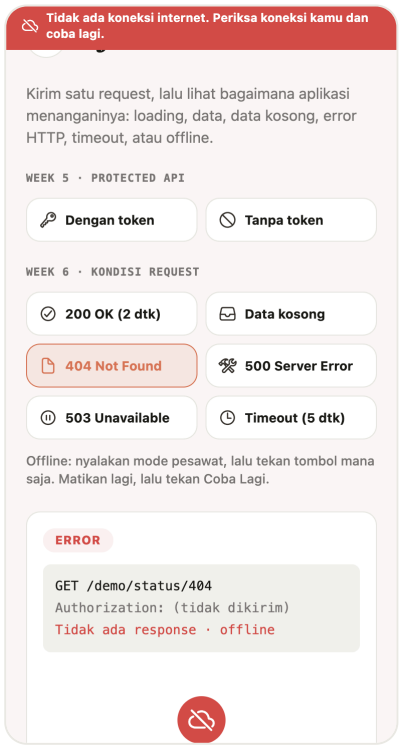
Di Creativo



404 → Data tidak ditemukan



Timeout → Waktu habis



Offline → banner + Coba Lagi

- **Task 01 · Handle API error**: `response.ok` dan status code selalu dicek; tiap status punya jenis dan pesan sendiri; data tidak ditampilkan saat gagal.
- **Task 02 · Loading, error & retry**: `useRequest` (idle → loading → success / empty / error); tombol **Coba Lagi** menjadi “Mencoba lagi…”.
- **Task 03 · Offline**: sebelum request, koneksi dicek dengan `expo-network`. Tanpa koneksi, request tidak dikirim dan banner merah muncul. Setelah request gagal, dicek lagi untuk membedakan “tidak ada internet” dan “server tidak bisa dihubungi”.
- **Uji Kondisi API** (Pengaturan → Uji kondisi API): tombol 200, data kosong, 404, 500, 503, timeout, dengan/tanpa token. Server punya endpoint demo `/demo/status/:code`, `/demo/empty`, `/demo/slow`.

Hubungan syntax

Loading → success / error (state request)

Di slide (PresidentKu)

```
const fetchPrograms = async () => {
  setLoading(true);
  setError(null);
  try {
    const response = await fetch(API_URL);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    const data = await response.json();
    setData(data);
  } catch (error) {
    setError(error.message || 'Terjadi kesalahan');
  } finally {
    setLoading(false);
  }
};
```

Di Creativo `src/controllers/useRequest.ts:33–43, 66–76`

```
const start = useCallback(async (retry: boolean) => {
  latest.current?.abort();
  const controller = new AbortController();
  latest.current = controller;
  dispatch({ type: 'start', retry });
  const result = await sendRef.current(controller.signal);
  // Replaced by a newer request, or the screen was closed
  if (controller.signal.aborted) return;
  if (result.error) dispatch({ type: 'failure', error: result.error });
  else dispatch({ type: 'success', data: result.data, empty: isEmptyRef.current(result.data) });
}, []);

export function requestReducer<T>(state: RequestState<T>, action: RequestAction<T>): RequestState<T> {
  switch (action.type) {
    case 'start':
      return { status: 'loading', data: null, error: null, attempt: action.retry ? state.attempt + 1 :
0 };
    case 'success':
      return { status: action.empty ? 'empty' : 'success', data: action.data, error: null, attempt:
state.attempt };
    case 'failure':
      // A failed request never shows data
      return { status: 'error', data: null, error: action.error, attempt: state.attempt };
  }
}
```

Status code → jenis error

Di slide (PresidentKu)

```
// 4xx: kesalahan client · 5xx: kesalahan server
throw new Error(`HTTP ${response.status}`);
```

Di Creativo — src/services/http.ts:76–89

```
function kindForStatus(status: number, detailed: boolean): ApiErrorKind {
  if (status === 404) return 'not_found';
  if (status === 429) return 'rate_limited';
  if (status >= 500) return 'server';
  if (detailed) {
    if (status === 400 || status === 422) return 'validation';
    if (status === 401) return 'unauthorized';
    if (status === 403) return 'forbidden';
    if (status === 409) return 'conflict';
    if (status === 413) return 'too_large';
  }
  return 'http';
}
```

Timeout

Di slide (PresidentKu)

```
// Timeout: batas waktu maksimal menunggu response
// misal 10–30 detik untuk setiap request
```

Di Creativo — src/services/http.ts:120–129

```
const timer = setTimeout(() => {
  timedOut = true;
  controller.abort();
}, timeoutMs);
const cancel = () => controller.abort();
// …
const abortKind = (): ApiErrorKind | null => (signal?.aborted ? 'aborted' : timedOut ? 'timeout' :
null);
```

Offline handling

Di slide (PresidentKu)

```
// CHECK CONNECTION → ONLINE? → API REQUEST
// tidak → OFFLINE STATE → USER FEEDBACK
```

Di Creativo — src/services/http.ts:138–149 · src/services/network.ts

```ts
// No connection at all: say so right away instead of waiting for the request to fail
if (!(await hasConnection())) return failure(apiError('offline'), `${method} ${url}`);
// Timed out or cancelled while the connection was being checked
if (controller.signal.aborted) return failure(apiError(abortKind() ?? 'aborted'), `${method} ${url}`);

let response: Response;
try {
  response = await fetch(url, init);
} catch (err) {
  // Network error: was it the phone's connection, or the server?
  return failure(apiError(abortKind() ?? ((await hasInternet()) ? 'network' : 'offline')), err);
}
```

Retry

Di slide (PresidentKu)

```
<Button title="Coba Lagi" onPress={fetchPrograms} />
```

Di Creativo — src/views/ui/ErrorState.tsx:52–60

```tsx
<Button
  label={retrying ? 'Mencoba lagi…' : 'Coba Lagi'}
  icon="refresh"
  size="sm"
  onPress={onRetry}
  loading={retrying}
  accessibilityHint="Mengirim ulang permintaan data"
  style={styles.action}
/>
```

Final checklist Sprint 05

- ✓ HTTP 2xx dipahami
- ✓ HTTP 4xx dipahami
- ✓ HTTP 5xx dipahami
- ✓ HTTP error handling
- ✓ Network error handling
- ✓ Loading state
- ✓ Success state
- ✓ Error state
- ✓ Empty state
- ✓ Timeout handling
- ✓ Retry mechanism
- ✓ Offline handling
- ✓ Offline feedback
- ✓ Retry setelah online
- ✓ Test successful request
- ✓ Test HTTP error
- ✓ Test network error
- ✓ Test timeout
- ✓ Test offline
- ✓ Test pada Expo Go
- □ Commit
- □ Push

Pertanyaan demo

Apa perbedaan HTTP error dan network error?

> HTTP error: server menjawab dengan 4xx/5xx. Network error: tidak ada jawaban sama sekali (server mati, alamat salah, koneksi putus).

Apa fungsi loading state?

> Memberi tahu bahwa request sedang berjalan sehingga aplikasi tidak terlihat hang.

Apa yang terjadi ketika API mengembalikan 404?

> Data tidak ditampilkan; muncul “Data tidak ditemukan” dan tombol Coba Lagi.

Apa yang terjadi ketika server mengembalikan 500?

> Muncul “Terjadi kesalahan pada server”; ini kesalahan sisi server, pengguna diminta mencoba lagi.

Apa yang terjadi ketika perangkat offline?

> Request tidak dikirim; muncul banner merah dan “Tidak ada koneksi internet. Periksa koneksi kamu dan coba lagi.”

Bagaimana mekanisme retry bekerja?

> Coba Lagi memanggil `retry()` yang mengirim request yang sama; state kembali ke loading (“Mencoba lagi…”) lalu berakhir di success, empty, atau error.

DEMO

PANDUAN PRESENTASI

Cara mendemokan Week 1–6

Persiapan, urutan presentasi ±10 menit, skrip tiap langkah, dan rencana cadangan. Fokus demo: **Flow → Result → Decision**.

Persiapan (sebelum kelas)

- ✓ `git pull` lalu `npm install`
- ✓ Terminal 2: `npx expo start`; scan QR dengan Expo Go
- ✓ Sidik jari sudah terdaftar di HP Android
- ✓ Terminal server terlihat di layar (log [auth] dan [api])
- ✓ `npm run server:reset`: data kembali ke 10 akun dummy
- ✓ HP dan laptop satu Wi-Fi (atau laptop memakai hotspot HP)
- ✓ `npm test` sekali: 68 passed
- ✓ Notifikasi HP dimatikan, kecerahan maksimal
- ✓ Terminal 1: `npm run server`; catat alamat Wi-Fi yang tercetak
- ✓ Di browser HP buka `http://IP-laptop:4000`: harus muncul JSON "Creativo Mock API"
- ✓ VS Code terbuka di file kunci (lihat tabel di bawah)
- ✓ Mirror layar HP ke proyektor (scrcpy / QuickTime) bila perlu

File yang dibuka saat menjelaskan	Untuk week
`src/app/(tabs)/home.tsx`	2 · elemen UI, responsif, accessibility
`src/app/(auth)/login.tsx` · `src/services/storage.ts`	3 · validasi, SecureStore
`server/data/users.json`	3 · akun baru + hash kata sandi
`src/services/http.ts`	4 & 6 · fetch, response.ok, timeout, offline
`src/services/api.ts` · `server/app.js`	5 · interceptor Bearer, validasi token

Urutan presentasi (±10 menit)

Menit	Bagian	Yang ditunjukkan
0–1	Week 1 · Pembukaan	Masalah, pengguna, solusi: Creativo
1–3	Week 2 · UI	Beranda, putar ke landscape, ukuran teks Besar
3–5	Week 3 · Auth & data lokal	Daftar → Masuk → validasi → biometrik → Keamanan & Sesi → Keluar
5–6	Week 4 · API	Lowongan: loading → data, ganti bidang, tarik untuk muat ulang
6–8	Week 5 · Secure API	Log token di terminal server, Dengan token (200) vs Tanpa token (401), logout
8–10	Week 6 · Reliable API	404, 500, data kosong, timeout, mode pesawat → offline → Coba Lagi
10+	Tanya jawab	Jawaban ada di bagian "Pertanyaan demo" tiap week

> Presentasi berkelompok: bagi per week (misalnya satu orang Week 1–2, satu orang Week 3, satu orang Week 4–5, satu orang Week 6). Setiap anggota harus bisa menjelaskan kontribusinya dan menunjukkan file kodenya.

Week 1 · Pembukaan

#	Lakukan	Ucapkan
1	Tampilkan slide atau README: product canvas	*“Karya para profesional tersebar di banyak platform dan CV tidak menunjukkan karya nyata. Creativo menyatukan karya dan koneksi profesional, supaya klien bisa menilai dari karya.”*
2	Tunjukkan GitHub: commit per minggu, template PR, CI	*“Kami bekerja per sprint. Setiap push dicek otomatis oleh CI: lint, typecheck, dan 68 tes.”*

Week 2 · Mobile UI

#	Lakukan	Ucapkan
1	Buka Beranda	*“Halaman utama punya header sapaan, judul, informasi utama, gambar dari karya terbaru, tombol Lihat Karya, menu cepat, dan bagian konten.”*
2	Putar HP ke landscape, lalu kembali	*“Layout responsif: lebar konten dibatasi, jumlah kolom karya mengikuti lebar layar, tidak ada yang keluar layar.”*
3	Profil → menu ≡ → Pengaturan → Ukuran teks: Besar → kembali ke Beranda	*“Untuk aksesibilitas teks bisa diperbesar. Ini data biasa, jadi disimpan di AsyncStorage.”*
4	(Opsional) Nyalakan TalkBack/VoiceOver, sentuh tombol Lihat Karya	*“Screen reader membaca label, peran tombol, dan hint-nya: Membuka feed karya.”*

Week 3 · Authentication & secure local data

#	Lakukan	Ucapkan
1	Halaman Sambutan → Daftar → tekan Daftar dengan form kosong	*“Validasi dilakukan dulu sebelum request: setiap kolom kosong diberi pesan.”*
2	Isi nama, email baru, kata sandi `Demo12345` → Daftar	*“Setelah daftar, saya diarahkan ke Masuk dengan email terisi. Alurnya Register → Login → Home seperti di modul.”*
3	Buka `server/data/users.json` di VS Code	*“Akun baru langsung tersimpan di sini. Kata sandinya hanya berupa hash scrypt, tidak ada teks aslinya.”*
4	Masuk dengan kata sandi salah	*“Credential salah ditolak dengan pesan umum, supaya tidak membocorkan akun mana yang terdaftar.”*
5	Masuk dengan `rinakartika` / `Creativo123` → muncul tawaran Sidik Jari → Aktifkan → scan	*“Login bisa dengan email atau username. Setelah login, login biometrik bisa diaktifkan.”*
6	Profil → Keamanan & Sesi	*“Token tersimpan di SecureStore. Data sensitif dan data biasa dipisah: token di SecureStore, pengaturan di AsyncStorage. Kata sandi tidak pernah disimpan di HP.”*
7	Keluar → konfirmasi → halaman Masuk → tekan Sidik Jari → scan	*“Logout menghapus session. Sidik jari tidak dikirim ke server: HP hanya memastikan cocok, lalu token biometrik dari SecureStore ditukar dengan access token baru.”*

Week 4 · Data dari API

#	Lakukan	Ucapkan
1	Beranda → menu cepat Lowongan	*“Data diambil dari Himalayas Jobs API dengan fetch. Saat menunggu tampil loading, lalu status Data berhasil dimuat.”*
2	Ganti chip bidang, lalu tarik ke bawah untuk memuat ulang	*“Setiap perubahan mengirim request baru; JSON diolah menjadi daftar kartu.”*
3	Tunjukkan `src/services/http.ts` baris 143–167	*“fetch → cek response.ok → response.json(). Kalau gagal, pesan error yang jelas, bukan crash.”*

Week 5 · Secure API

#	Lakukan	Ucapkan
1	Login, lalu tunjuk terminal server	*“Server memverifikasi kredensial lalu membuat access token JWT HS256 yang berlaku 60 menit.”*
2	Keamanan & Sesi: Format, Peran, Berlaku s/d, Disimpan di	*“Token disimpan aman di SecureStore dan dikirim sebagai Authorization: Bearer di setiap request.”*
3	Uji protected API → Dengan token, lalu Tanpa token	*“Dengan token: 200 dan data tampil. Tanpa token: 401, data tidak diberikan.” (tunjuk log server: ditolak: tanpa token)*
4	Tunjukkan `src/services/api.ts` baris 73–74	*“Ini interceptor: token dibaca dari SecureStore lalu ditambahkan otomatis ke header, jadi tidak ditulis berulang.”*
5	Keluar → tunjuk terminal server	*“Logout mencabut token di server lalu menghapusnya dari HP. Token lama tidak bisa dipakai lagi.”*

Week 6 · Error handling & offline

#	Lakukan	Ucapkan
1	Pengaturan → Uji kondisi API → 404, lalu 500	*“Aplikasi memeriksa status code. 404 berarti data tidak ditemukan, 500 berarti kesalahan di server. Data tidak ditampilkan saat gagal.”*
2	Data kosong	*“Request berhasil tetapi tidak ada data: tampil empty state, bukan error.”*
3	Timeout (5 dtk) → tunggu → Coba Lagi	*“Server sengaja lambat. Setelah batas waktu request dihentikan, jadi aplikasi tidak terlihat hang.”*
4	Nyalakan mode pesawat → tekan tombol mana saja (atau tarik Beranda)	*“Tanpa koneksi, request tidak dikirim. Muncul banner merah dan pesan Tidak ada koneksi internet.”*
5	Matikan mode pesawat, tunggu Wi-Fi tersambung → Coba Lagi	*“Setelah online lagi, retry mengirim ulang request dan data berhasil dimuat.”*
6	Tunjukkan hasil `npm test`	*“Semua skenario ini juga diuji otomatis: 68 tes lulus.”*

Rencana cadangan

Masalah saat demo	Solusi
“Server tidak dapat dihubungi”	Pastikan `npm run server` berjalan dan HP satu Wi-Fi. Buka `http://IP:4000` di browser HP. Izinkan Node di firewall macOS. Wi-Fi kampus sering memblokir antarperangkat: pakai hotspot HP.
Alamat server berbeda / pakai `--tunnel`	Isi `EXPO_PUBLIC_API_URL=http://IP:4000` di `.env`, lalu `npx expo start -c`.
Face ID tidak muncul di iPhone	Batasan Expo Go: Face ID butuh development build. Demo biometrik di Android, atau jelaskan tile “Tidak tersedia di HP ini”.
Tile biometrik bilang “belum aktif”	Masuk dengan kata sandi, lalu Profil → Keamanan & Sesi → nyalakan Login biometrik.
Data berantakan setelah latihan	`npm run server:reset`, lalu login ulang.
Sesi habis (60 menit) saat demo	Itu fitur: jelaskan pesan “Sesi kamu telah berakhir”, lalu masuk dengan sidik jari.
Lowongan gagal (internet kampus)	Jadikan contoh error handling Week 4/6: pesan jelas + Coba Lagi.
Mode pesawat memutus Expo	Jangan tutup aplikasi selama mode pesawat; setelah Wi-Fi tersambung tekan Coba Lagi. Bila layar macet, tekan `r` di terminal Expo.

Lampiran: akun dummy

Semua akun memakai kata sandi `Creativo123`. Masuk bisa dengan email atau username. Data awal ada di `server/seed/`; `npm run server:reset` mengembalikan data ke kondisi ini (akun hasil daftar ikut terhapus).

#	Nama	Email	Username	Profesi
1	Rina Kartika	`rina@creativo.id`	`rinakartika`	Desainer UI/UX · Senior
2	Bagas Pratama	`bagas@creativo.id`	`bagaspratama`	Engineer Informatika · Menengah
3	Dewi Lestari	`dewi@creativo.id`	`dewilestari`	Fotografer · Senior
4	Andi Saputra	`andi@creativo.id`	`andisaputra`	Arsitek · Lead
5	Maya Anggraini	`maya@creativo.id`	`mayaanggraini`	Desainer Grafis · Menengah
6	Fajar Nugroho	`fajar@creativo.id`	`fajarnugroho`	Data Scientist · Menengah
7	Nadia Putri	`nadia@creativo.id`	`nadiaputri`	Kreator Video & Konten · Junior
8	Reza Mahendra	`reza@creativo.id`	`rezamahendra`	Digital Marketer · Senior
9	Intan Permata	`intan@creativo.id`	`intanpermata`	Engineer Informatika · Mahasiswa
10	Yoga Firmansyah	`yoga@creativo.id`	`yogafirmansyah`	Kreator Video & Konten · Menengah

Data disimpan di mana?

Data	Lokasi	Alasan
Akun (nama, email, hash kata sandi)	`server/data/users.json`	Milik server, dibaca saat login
Postingan, suka, komentar, pesan, notifikasi	`server/data/db.json`	Dibagi semua pengguna
Access token, token biometrik	SecureStore di HP	Sensitif
Ukuran teks, tab feed terakhir	AsyncStorage di HP	Data biasa
Kata sandi	Tidak disimpan di HP	Hanya dikirim saat login

Hasil tes

```
PASS src/__tests__/reliable.test.ts     (Week 6)
PASS src/__tests__/security.test.ts     (Week 3)
PASS src/__tests__/token.test.ts        (Week 5)
PASS src/__tests__/api.test.ts          (Week 4)
Test Suites: 4 passed, 4 total
Tests:       68 passed, 68 total
```